

M06 Module Review

Git + GitHub + Linux/WSL Engineering Foundations | Review after attempt

Review rule

Open only after a genuine attempt. A review is diagnostic material, not the first way to solve the task.

M06-L01 - Verify WSL2 and the Linux engineering workspace

Strong evidence: You can open WSL, locate the workspace and state the Windows-vs-Linux path boundary without guessing.

Repair focus: Reinstalling or running admin commands because a tutorial says so without first verifying the existing environment.

M06-L02 - Linux navigation and safe file operations

Strong evidence: You can predict the exact target of a file command before running it.

Repair focus: Running rm against the wrong directory or using a copied recursive command without understanding its expansion.

M06-L03 - Pipes, redirection and data inspection

Strong evidence: You can explain what flows through the pipe and where redirected output goes.

Repair focus: Treating shell line counts as business-row counts without accounting for headers or quoted multiline records.

M06-L04 - Permissions, packages, sudo and command safety

Strong evidence: You can explain why a command needs elevation before using sudo.

Repair focus: Using sudo for routine project commands and accidentally creating root-owned project files.

M06-L05 - Git repository state: init, status, add, commit and log

Strong evidence: You can identify working-tree, staged and committed changes from git status/diff.

Repair focus: Running git add . and committing unknown generated files or secrets because status was not inspected.

M06-L06 - Branches, merges and conflict resolution

Strong evidence: You can resolve a fresh conflict without copying a memorized sequence.

Repair focus: Deleting conflict markers without checking which business content should survive.

M06-L07 - GitHub remotes: push, fetch, pull and tracking

Strong evidence: You can explain fetch vs pull vs push from the direction of data and resulting repository state.

Repair focus: Pulling with unresolved local changes or assuming fetch automatically changes the working tree.

M06-L08 - Pull requests, review and collaboration workflow

Strong evidence: You can explain the change intent, review the diff and synchronize after merge.

Repair focus: Mixing unrelated changes into one PR or merging without reading the final diff.

M06-L09 - Python virtual environments inside the repository

Strong evidence: You can rebuild the environment after deleting .env using version-controlled dependency information.

Repair focus: Committing .env or assuming a copied environment is portable across machines.

M06-L10 - .gitignore, secrets and repository hygiene

Strong evidence: You can prove secret/runtime files are not tracked and state the remediation boundary for an exposed credential.

Repair focus: Assuming adding a filename to .gitignore removes a secret that was already committed.

M06-L11 - Diff, restore, revert and safe history repair

Strong evidence: Given a repository mistake, you can choose restore/revert/history rewrite and explain why.

Repair focus: Using git reset --hard or force push as a first response to a mistake without preserving/understanding work.

M06-L12 - Shell scripts, exit codes and environment-driven commands

Strong evidence: You can run the script from a fresh shell and interpret its exit status.

Repair focus: Writing a script that only works from one current directory or continues after a failed command while still printing success.

M06-L13 - Repository handoff, reproducibility and evidence

Strong evidence: A fresh clone can be run and validated without relying on your memory or hidden files.

Repair focus: A repository that works only because of untracked local state or undocumented setup steps.